



## Employee Hierarchy – Technical Document

Dandré Nel

September 2026

Live site: [employees.dandrenel.co.za](https://employees.dandrenel.co.za)

Repository: [github.com/DandreNel7/employee\\_hierarchy](https://github.com/DandreNel7/employee_hierarchy)

## Table of Contents

|                                                                                                                                  |          |
|----------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>Employee Hierarchy - Technical Document .....</b>                                                                             | <b>1</b> |
| <b>Dandré Nel .....</b>                                                                                                          | <b>1</b> |
| <b>September 2026 .....</b>                                                                                                      | <b>1</b> |
| <b>Live site: <a href="https://employees.dandrenel.co.za">employees.dandrenel.co.za</a>.....</b>                                 | <b>1</b> |
| <b>Repository: <a href="https://github.com/DandreNel7/employee_hierarchy">github.com/DandreNel7/employee_hierarchy</a> .....</b> | <b>1</b> |
| <b>1. Overview .....</b>                                                                                                         | <b>3</b> |
| <b>2. Architecture .....</b>                                                                                                     | <b>3</b> |
| <b>3. Data model.....</b>                                                                                                        | <b>4</b> |
| <b>4. Design patterns .....</b>                                                                                                  | <b>5</b> |
| <b>5. API.....</b>                                                                                                               | <b>5</b> |
| <b>6. Technologies and justification .....</b>                                                                                   | <b>6</b> |
| <b>7. Security .....</b>                                                                                                         | <b>7</b> |
| <b>8. Deployment and CI/CD.....</b>                                                                                              | <b>8</b> |
| <b>AWS services used .....</b>                                                                                                   | <b>8</b> |
| <b>The pipeline .....</b>                                                                                                        | <b>8</b> |
| <b>Configuration and secrets.....</b>                                                                                            | <b>9</b> |
| <b>9. Testing .....</b>                                                                                                          | <b>9</b> |
| <b>10. Additional functionality .....</b>                                                                                        | <b>9</b> |

# 1. Overview

A web application for managing EPI-USE Africa's employees and their reporting structure. An administrator can add, edit and delete employees, set each person's reporting line manager, view the organisation as an interactive chart, and sort, filter and export a table of everyone.

Everything is stored in a PostgreSQL database on Amazon RDS. The application holds no hardcoded or file based data. The initial employee records were loaded through the application's own CSV import feature, which writes through the same API as any other change.

# 2. Architecture

The application is a React single page app talking to a Python REST API, both running as Docker containers on a single EC2 instance behind CloudFront.

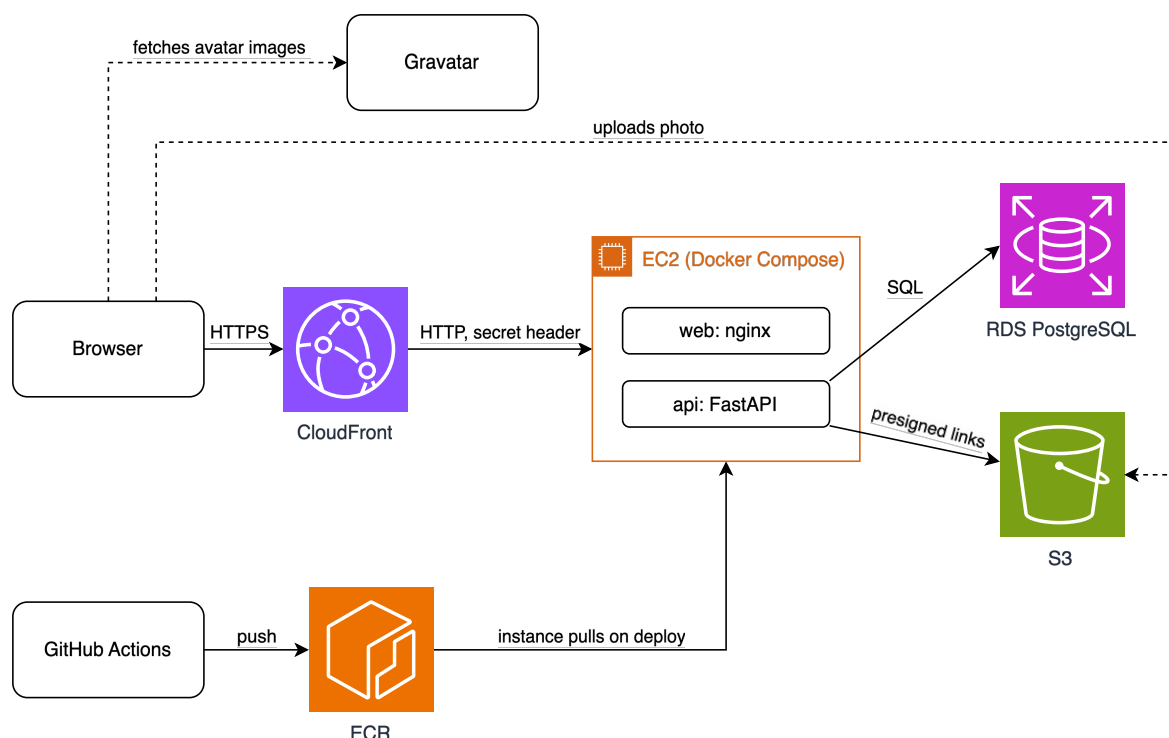


Figure 1: The browser reaches CloudFront over HTTPS, which forwards to nginx on the EC2 instance. nginx serves the built frontend and proxies /api to the FastAPI container, which talks to RDS and S3. Avatar images are fetched by the browser directly from Gravatar, and photos are uploaded straight to S3.

nginx serves the frontend and proxies /api to the API container, so the browser sees a single origin: no CORS configuration, and one place to set security headers. CloudFront terminates HTTPS on the custom domain and is the only route to the instance.

The API itself is layered:

| Layer                                      | Responsibility                                                  |
|--------------------------------------------|-----------------------------------------------------------------|
| Routers ( <a href="#">app/routers/</a> )   | HTTP only: read the request, call a service, return a response  |
| Services ( <a href="#">app/services/</a> ) | Business rules: hierarchy checks, CSV import, storage, Gravatar |
| Models ( <a href="#">app/models/</a> )     | Tables and relationships                                        |

### 3. Data model

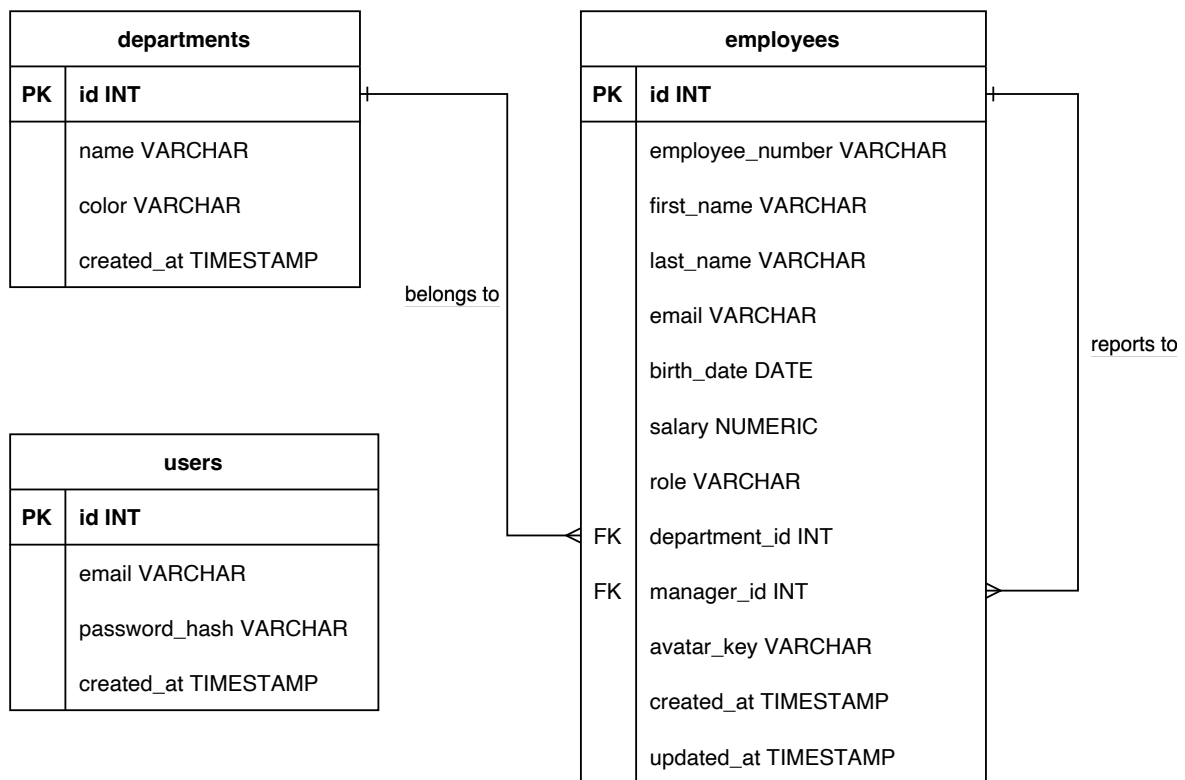


Figure 2: Three tables. The self-referencing *manager\_id* is what forms the hierarchy.

A null *manager\_id* means the employee is at the top of the hierarchy, so the CEO is an employee with no manager. More than one employee may sit at the top.

| Constraint                                                                | Purpose                                                                    |
|---------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <code>CHECK (manager_id &lt;&gt; id)</code>                               | An employee cannot be their own manager, enforced by the database          |
| <code>CHECK (salary &gt;= 0)</code>                                       | No negative salaries                                                       |
| <code>UNIQUE (email), UNIQUE (employee_number)</code>                     | No duplicates                                                              |
| <code>employee_number DEFAULT 'EMP-'    lpad(nextval(...), 4, '0')</code> | Numbers generated by PostgreSQL, so they cannot be duplicated or forgotten |
| Indexes on <i>manager_id</i> , <i>department_id</i>                       | Fast subtree and filter queries                                            |

`users` holds administrator logins and is separate from `employees`.

The hierarchy is stored as an adjacency list: `manager_id` on each row. A move is a single update, and subtree questions, used to detect loops, are answered with a recursive query.

## 4. Design patterns

| Pattern                        | Where                             | Why                                                                                                                                                                                            |
|--------------------------------|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Layered architecture           | routers / services / models       | Rules are testable without HTTP                                                                                                                                                                |
| Dependency injection           | <code>app/deps.py</code>          | Routes are given a database session and the current user.<br><code>dependencies=[ Depends(get_current_user)]</code> on the router means a new endpoint cannot accidentally be left unprotected |
| Data transfer objects          | <code>app/schemas/</code>         | Separate in and out models, so a password hash can never be returned and <code>avatar_url</code> can be computed without being a column                                                        |
| Unit of work                   | SQLAlchemy session                | A failed CSV import leaves nothing behind                                                                                                                                                      |
| Adjacency list                 | <code>employees.manager_id</code> | Simple moves, recursive CTE for subtrees                                                                                                                                                       |
| Reverse proxy                  | nginx                             | One origin, no CORS, one place for security headers                                                                                                                                            |
| Presigned URLs                 | <code>services/storage.py</code>  | Image data never passes through the API                                                                                                                                                        |
| Configuration from environment | <code>app/config.py</code>        | The same image runs locally and in production                                                                                                                                                  |
| Strategy                       | Table column definitions          | Each column supplies its own filter function: text, faceted list or numeric range                                                                                                              |

## 5. API

All routes are under `/api`. Every route except login requires a valid session cookie.

| Method | Route                                         | Purpose                                        | Success |
|--------|-----------------------------------------------|------------------------------------------------|---------|
| POST   | <code>/auth/login</code>                      | Sign in, sets the session cookie               | 200     |
| POST   | <code>/auth/logout</code>                     | Clear the cookie                               | 204     |
| GET    | <code>/auth/me</code>                         | Who is signed in                               | 200     |
| GET    | <code>/employees</code>                       | Every employee, with department and avatar URL | 200     |
| GET    | <code>/employees/{id}</code>                  | One employee                                   | 200     |
| POST   | <code>/employees</code>                       | Create                                         | 201     |
| PUT    | <code>/employees/{id}</code>                  | Update                                         | 200     |
| DELETE | <code>/employees/{id}?reassign_to={id}</code> | Delete, moving any direct reports              | 204     |
| POST   | <code>/employees/{id}/avatar</code>           | Request a presigned upload form                | 200     |

| Method      | Route                  | Purpose                                      | Success  |
|-------------|------------------------|----------------------------------------------|----------|
| PUT         | /employees/{id}/avatar | Save the uploaded photo against the employee | 200      |
| DELETE      | /employees/{id}/avatar | Remove the photo, falling back to Gravatar   | 200      |
| POST        | /employees/import      | Bulk import from CSV                         | 200      |
| GET, POST   | /departments           | List, create                                 | 200, 201 |
| PUT, DELETE | /departments/{id}      | Update, delete                               | 200, 204 |
| GET         | /stats                 | Dashboard figures                            | 200      |

Errors are consistent, so the interface can show the reason directly to the user.

| Status | Meaning                               | Example message                                                                   |
|--------|---------------------------------------|-----------------------------------------------------------------------------------|
| 400    | A business rule was broken            | "Sipho Dlamini already reports to Thandi Mokoena, so they cannot be the manager." |
| 401    | Not signed in, or the session expired | "Not logged in"                                                                   |
| 404    | No such record                        | "Employee not found"                                                              |
| 422    | The request failed validation         | Field-level detail from Pydantic                                                  |
| 429    | Too many login attempts               | Rate limit exceeded                                                               |

GET /employees returns the whole list, the browser builds the chart and filters the table locally.

Photo uploads bypass the application server: the browser asks the API for a short lived upload form, sends the file straight to S3, then tells the API which key was used.

The API sets the limits: JPEG, PNG or WebP, 2 MB, five minutes to use the form, and the storage key itself.

## 6. Technologies and justification

| Choice                    | Why                                                                                                                                                                                                                            |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| React + TypeScript (Vite) | Types catch mistakes across the Employee model before runtime. Vite builds in seconds                                                                                                                                          |
| Tailwind + shadcn/ui      | Components are generated into the repository rather than installed, so they were restyled to EPI-USE colours by changing CSS variables once. Built on Radix, so keyboard navigation and screen reader support come as standard |

|                             |                                                                                                                                                                                         |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TanStack Table / React Flow | Headless: they provide sorting, filtering and graph behaviour but render nothing, so the design system stays consistent                                                                 |
| dagre                       | Lays the chart out automatically, so it stays readable no matter how the hierarchy changes                                                                                              |
| FastAPI                     | Validation through Pydantic, dependency injection built in, and OpenAPI generated from the code                                                                                         |
| SQLAlchemy + Alembic        | Schema changes are versioned in the repository and applied automatically on deploy, so the database cannot drift from the code                                                          |
| PostgreSQL on RDS           | Relational data with a self-referencing foreign key. Managed backups, patching and encryption.                                                                                          |
| AWS                         | One provider sharing a network boundary, so the database has no public address and no credentials are stored anywhere. I have worked with it before and was comfortable deploying on it |
| Docker + GitHub Actions     | The same image runs locally and in production. every push to main builds, tests and deploys                                                                                             |

## 7. Security

The application is on a public URL and holds salary and personal information.

- **Login.** Passwords hashed with bcrypt. The session is a JWT in an httpOnly, Secure, SameSite=Lax cookie, so JavaScript cannot read it. Failed logins return an identical message whether or not the email exists, and are limited to five attempts per minute per address.
- **Authorisation.** Every route except login is behind a server side dependency. Hiding buttons in the interface is convenience only.
- **Input.** All request bodies are validated by Pydantic. Database access is through SQLAlchemy with bound parameters, including the recursive CTE, so there is no SQL built from strings.
- **Output.** React escapes by default. CSV exports prefix cells beginning with `=`, `+`, `-` or `@` with a quote, so a value cannot become a formula when opened in Excel.
- **Headers.** Content Security Policy, HSTS, `nosniff`, `X-Frame-Options: DENY` and a referrer policy, all set in nginx.
- **Network.** The database has no public access and accepts connections only from the application's security group. The instance accepts traffic only from CloudFront's address ranges, and additionally rejects any request without a secret header that CloudFront adds. There is no SSH port. administration is through AWS Session Manager.

- **Credentials.** No AWS access keys exist anywhere. The instance uses an IAM role scoped to one S3 bucket, and GitHub Actions authenticates through OIDC, no long lived secret is stored in the repository.

## 8. Deployment and CI/CD

### AWS services used

| Service                           | Role in the solution                                                                     |
|-----------------------------------|------------------------------------------------------------------------------------------|
| EC2 (t3.small)                    | Runs the two containers through Docker Compose                                           |
| RDS (PostgreSQL 17, db.t4g.micro) | The database. No public access, encrypted                                                |
| S3                                | Uploaded profile photos. Private bucket, reached only through presigned URLs             |
| ECR                               | Stores the two Docker images                                                             |
| CloudFront                        | HTTPS on the custom domain, caches static assets, hides the instance                     |
| ACM                               | The TLS certificate, renewed automatically                                               |
| IAM                               | An instance role scoped to one bucket, and a deployment role GitHub assumes through OIDC |
| Systems Manager                   | Shell access and remote commands without opening an SSH port                             |

### The pipeline

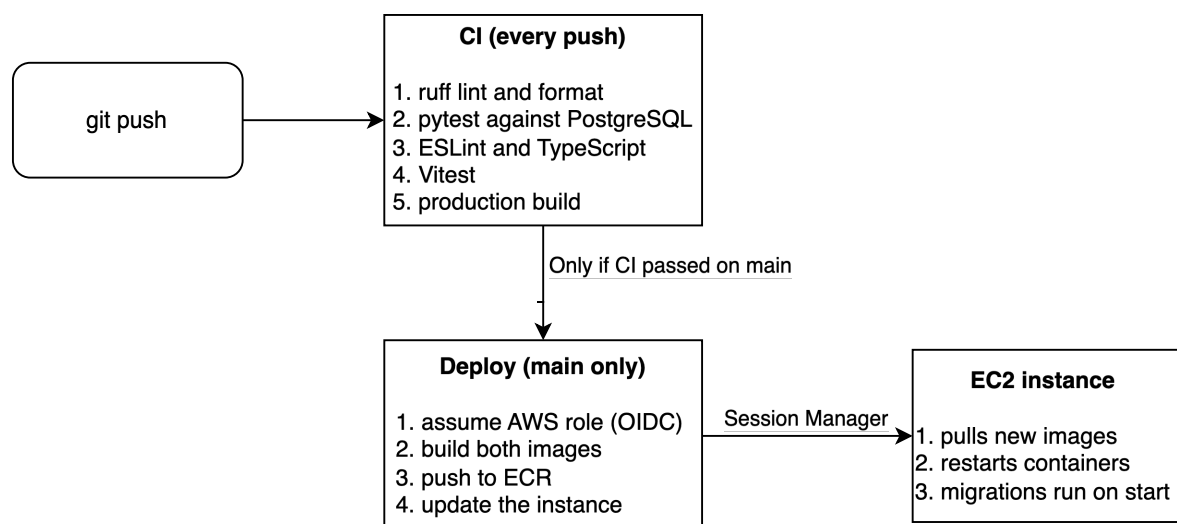


Figure 3: Every push runs the checks. Only *main* deploys.

CI runs on every push and pull request, against a real PostgreSQL service container rather than a stand-in, so the tests exercise the same database engine used in production.

Deploy runs only after CI has completed successfully on **main**, so a failing build is never released.

## Configuration and secrets

Application settings come from an environment file on the instance, readable only by root, and are never committed. **.env.example** documents which values are required. GitHub holds only what is needed to deploy: the role to assume, the registry address and the instance identifier. The database password and signing key exist nowhere except on the instance.

## 9. Testing

37 automated tests, covering the rules most likely to break.

- **Backend** (18, pytest). Login behaviour including the identical failure message and the rate limit. Employee validation, and the hierarchy rules: self-management, reporting to your own report, loops further down the chain, and each branch of delete-and-reassign. The schema is built by running the real migrations, so migrations are tested too.
- **Frontend** (19, Vitest). The tree helpers that build the chart, the descendant calculation that excludes invalid managers from the dropdown, and the CSV formula-injection guard.

## 10. Additional functionality

Built beyond the minimum requirement.

| Feature                       | What it does                                                                                                                                       |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Circular reporting prevention | Rejects any change that would create a loop, at any depth, not only self-management                                                                |
| Delete with reassignment      | Choosing where a manager's team goes, including promoting one of their own reports                                                                 |
| Departments                   | Colour coded, filterable, and protected from deletion while employees are assigned                                                                 |
| CSV import                    | Validates every row first, including duplicate emails, unknown managers and loops within the file. nothing is saved unless the whole file is valid |
| CSV export                    | Exports the filtered view, quoting cells that Excel would otherwise treat as formulas                                                              |
| Photo upload                  | Uploads to S3 and shows the photo in place of the Gravatar                                                                                         |
| Dashboard                     | Headcount, department and salary figures, with charts by department and role                                                                       |
| Authentication                | Admin login with bcrypt, an httpOnly session cookie and rate limited attempts                                                                      |
| Engineering                   | Docker, CI/CD, database migrations, automated tests, linting on both sides                                                                         |
| Infrastructure                | Custom domain with HTTPS, private database, security headers, no stored credentials                                                                |